Guion Metodológico de Sustentación y Dossier de Evidencias de Software

Procedimiento integral de arranque, guion de defensa oral paso a paso, detalle explicativo de los 39 tests de PHPUnit y catálogo de 12 evidencias auditadas.

Ficha Resumen para la Sustentación Oral y Auditoría:

Rol del Alumno: Estudiante de Sistemas / QA Tester
Asignatura: Testing y Calidad de Software
Total de Tests: 39 tests • 140 aserciones (100% OK)
Tiempo de Ejecución: 0.21 segundos en RAM (:memory:)
Dossier de Evidencias: 12 Figuras (Web, CLI y VS Code)
Normativa Evaluada: SUNAT Resolución 097-2012 / UBL 2.1

Fase 0: Cómo Arrancar el Proyecto Desde Cero Frente al Profesor

No des por hecho que el sistema ya está corriendo. Si el profesor te pide levantarlo desde cero en vivo, sigue estas rutas y comandos exactos:

Rutas Principales en tu Computadora (Windows):
- **Carpeta Raíz del Proyecto:** `D:\Escritorio edobia-web\Billing`
- **Carpeta del Backend (PHP):** `D:\Escritorio edobia-web\Billing\backend`
- **Carpeta del Frontend (React):** `D:\Escritorio edobia-web\Billing\frontend`

Tip de Windows: *Entra a cualquier carpeta en el Explorador de Archivos, haz un clic en la barra de ruta superior, escribe **powershell** y presiona **Enter**. Se abrirá la consola exactamente en esa ruta.*

1. Terminal 1 — Servidor Backend PHP (Puerto 8000)

Abrir PowerShell en: `D:\Escritorio edobia-web\Billing\backend`

```
php -S 127.0.0.1:8000 -t public
```

QUÉ DECIR AL PROFESOR:
*"Profesor, en esta primera terminal nos ubicamos en la carpeta **backend** y levantamos el servidor PHP en el puerto 8000. Este servidor atiende el motor tributario, la firma digital XMLDSIG de comprobantes y la base de datos SQLite."*

2. Terminal 2 — Servidor Frontend React Vite (Puerto 5173)

Abrir PowerShell en: `D:\Escritorio edobia-web\Billing\frontend`

```
npm run dev
```

QUÉ DECIR AL PROFESOR:
*"En esta segunda terminal nos ubicamos en la carpeta **frontend** y levantamos la interfaz web con React y Vite en el puerto 5173, que es la pantalla interactiva del cajero."*

3. Abrir el Navegador Web (Chrome o Edge)

Ver Figuras 1, 2, 3, 4 en Págs. 7-10

Ingresar a la URL: `http://localhost:5173`

QUÉ DECIR AL PROFESOR:
*"Abrimos el navegador en **localhost:5173**. Como se aprecia en la **Figura 1 de nuestro Dossier (Pág. 7)**, el facturador está conectado en vivo con el backend. También podemos mostrar el inventario con Kardex (**Figura 2, Pág. 8**), el historial de comprobantes (**Figura 3, Pág. 9**) y el visor interactivo del XML firmado con UBL 2.1 (**Figura 4, Pág. 10**)."*

4. Terminal 3 — Consola de Pruebas Automatizadas PHPUnit 10

Ver Figuras 5 y 7 en Págs. 11 y 13

Abrir PowerShell en: `D:\Escritorio edobia-web\Billing\backend`

```
vendor/bin/phpunit --testdox --colors=never
```

QUÉ DECIR AL PROFESOR:
*"Y en esta tercera terminal corremos los 39 tests de PHPUnit. Como se evidencia en la **Figura 7 (Pág. 13)**, la terminal arroja 140 aserciones en verde en 0.21 segundos. Esta misma ejecución la podemos disparar desde la interfaz web, como se observa en la **Figura 5 (Pág. 11)**."*

Fase 1: Mapa General de los 39 Tests (¿Cuántos son y cómo se dividen?)

Para que el profesor vea que dominas la teoría de QA, explícale que los **39 tests** no están tirados al azar, sino organizados en **6 suites de prueba** bajo la **Pirámide de Pruebas de Mike Cohn**:

Nivel Pirámide	Suite y Archivo	Cantidad	¿Qué tipo de pruebas hace?	Evidencia
1. Unitaria	ValueObjectsTest.php	**14 tests**	Valida RUCs con fórmula Módulo 11, DNIs de 8 dígitos y series F001/B001.	**Figura 9 (Pág. 15)**
1. Unitaria	CalculadoraTributariaTest.php	**6 tests**	Valida el 18% de IGV, exonerados, inafectos, bolsas ICBPER y detracción SPOT.	**Figura 8 (Pág. 14)**
1. Unitaria	ComprobanteDomainRulesTest.php	**6 tests**	Reglas SUNAT 2017 (Factura a DNI) y 2511 (Boleta > S/ 700), y bajas de 7 días.	**Figura 6 (Pág. 12)**
2. Integración	XmlAndCryptoIntegrationTest.php	**4 tests**	Verifica la estructura XML UBL 2.1, firma SHA-256 y detección de adulteraciones.	**Figura 10 (Pág. 16)**
2. Integración	SqlitePersistenceIntegrationTest.php	**3 tests**	Base de datos SQLite en memoria, descuento de stock y tabla de idempotencia.	**Figura 12 (Pág. 18)**
3. Feature	FacturacionElectronicaMocksFeatureTest.php	**6 tests**	Simula respuestas de SUNAT con Mocks, caídas de red y rollback de Kardex.	**Figura 11 (Pág. 17)**
TOTAL GENERAL DE PRUEBAS AUTOMATIZADAS		**39 tests**	**140 aserciones verificadas al 100% en 0.21 segundos (Acta en Pág. 19)**	

Fase 2: Detalle de Cada Uno de los 39 Tests (De qué trata y por qué se hizo)

Suite Unitaria 1: ValueObjectsTest.php (14 Tests) **Auditado en Figura 9 • Pág. 15**

Ubicación: `backend/tests/Unit/ValueObjectsTest.php`

> **QUÉ DECIR AL PROFESOR AL EXPLICAR ESTA SUITE:**
> *"Profesor, en esta primera suite comprobamos que ninguna entidad de negocio se cree con datos falsos. Como se muestra en la **Figura 9 del Dossier (Pág. 15)** en Visual Studio Code, auditamos el archivo `src/Domain/ValueObjects/Ruc.php` donde implementamos los factores oficiales de SUNAT `[5, 4, 3, 2, 7, 6, 5, 4, 3, 2]` con residuo Módulo 11 para rechazar RUCs inventados antes de tocar la base de datos."*

#	Nombre del Test	¿De qué trata y por qué se hizo?
1	testAceptaRucsValidos (SUNAT 20)	**De qué trata:** Prueba el RUC 20131312955 (Persona Jurídica). **Por qué se hizo:** Para asegurar que el algoritmo reconozca que empieza con 20 y que su dígito verificador es correcto.
2	testAceptaRucsValidos (BCP 20)	**De qué trata:** Prueba el RUC 20100047218 del Banco de Crédito. **Por qué se hizo:** Valida que las empresas bancarias pasen la validación matemática sin trabas.
3	testAceptaRucsValidos (Alicorp 20)	**De qué trata:** Prueba el RUC 20100055237 de Alicorp SAA. **Por qué se hizo:** Confirma que los RUCs corporativos industriales sean aceptados.
4	testAceptaRucsValidos (Persona 10)	**De qué trata:** Prueba el RUC 10456789124 (Persona Natural con Negocio). **Por qué se hizo:** Para validar que el sistema diferencie a personas con negocio de empresas jurídicas.
5	testRechazaRucsInvalidos (< 11 dígitos)	**De qué trata:** Envía un RUC con solo 10 dígitos (ej. 2013131295). **Por qué se hizo:** Evita que un cajero guarde un RUC incompleto por error tipográfico.
6	testRechazaRucsInvalidos (> 11 dígitos)	**De qué trata:** Envía un RUC de 12 dígitos (201313129559). **Por qué se hizo:** SUNAT rechaza cualquier comprobante con documentos de longitud incorrecta.
7	testRechazaRucsInvalidos (Con letras)	**De qué trata:** Envía un RUC que tiene una letra 'A' (2013131295A). **Por qué se hizo:** Garantiza que el campo sea estrictamente numérico y previene inyecciones de datos.
8	testRechazaRucsInvalidos (Prefijo 99)	**De qué trata:** Envía un RUC que empieza con 99 (99131312955). **Por qué se hizo:** En Perú solo existen RUCs que empiezan con 10, 15, 17 o 20. El prefijo 99 no existe.
9	testRechazaRucsInvalidos (Módulo 11 adulterado)	**De qué trata:** Envía 20131312954 donde cambiamos el 5 final por un 4. **Por qué se hizo:** Demuestra que el algoritmo detecta si alguien inventa un RUC con números al azar.
10	testRechazaRucsInvalidos (Módulo 11 en RUC 10)	**De qué trata:** Envía un RUC 10 con el dígito verificador adulterado. **Por qué se hizo:** Protege que no se emitan facturas a personas naturales con RUCs inventados.
11	testValidacionDeDni	**De qué trata:** Valida que el DNI tenga 8 dígitos y rechaza DNI puros ceros (00000000). **Por qué se hizo:** Para cumplir la norma de RENIEC y evitar boletas con DNIs falsos.
12	testFormateoDeSerieYCorrelativo	**De qué trata:** Envía serie 'f001' y número 45, y verifica que formatee 'F001-00000045'. **Por qué se hizo:** SUNAT exige que el correlativo tenga 8 dígitos rellenos con ceros a la izquierda.
13	testRechazaIncoherenciaEntreSerieYTipo	**De qué trata:** Intenta crear una Factura con la serie B001 (que es de Boleta). **Por qué se hizo:** SUNAT prohíbe mezclar series; Facturas van con 'F' y Boletas con 'B'.
14	testMonedaExtranjeraExigeTipoDeCambioValido	**De qué trata:** Exige que si facturas en Dólares (USD), ingreses un tipo de cambio real mayor a 1. **Por qué se hizo:** Evita emitir facturas en dólares sin tipo de cambio oficial, lo cual es causal de sanción.

Suite Unitaria 2: CalculadoraTributariaTest.php (6 Tests) Auditado en Figura 8 • Pág. 14

Ubicación: `backend/tests/Unit/CalculadoraTributariaTest.php`

QUÉ DECIR AL PROFESOR AL EXPLICAR ESTA SUITE:
*"En la suite de la Calculadora Tributaria auditamos la matemática fiscal. Como se muestra en la **Figura 8 del Dossier (Pág. 14)**, abrimos en Visual Studio Code `CalculadoraTributariaTest.php` y destacamos el método `testCalculaComprobanteMixtoComplejo`: demuestra que en una sola factura pueden coexistir bienes gravados con el 18% de IGV, libros exonerados por Ley 31053, muestras gratis y bolsas plásticas con ICBPER, sin que se contaminen las bases imponibles."*

#	Nombre del Test	¿De qué trata y por qué se hizo?
15	testCalculaOperacionGravadaEstandar	**De qué trata:** Vende 2 laptops de S/ 2500 y 5 mouses de S/ 50. **Por qué se hizo:** Comprueba la matemática básica: $5250 imes 18\% = S/ 945$ de IGV. Verifica que no haya redondeos erróneos.
16	testCalculaComprobanteMixtoComplejo	**De qué trata:** Junta productos con IGV, libros exonerados, premios de regalo y bolsas plásticas. **Por qué se hizo:** Valida que los impuestos no se mezclen: el IGV solo grava al monitor y las bolsas pagan su impuesto ICBPER aparte.
17	testCalculaDetraccionSpotEnDolares...	**De qué trata:** Factura un servicio de consultoría en dólares (USD 354 que son S/ 1345.20). **Por qué se hizo:** Valida la ley de detracciones SPOT: si el servicio supera S/ 700, retiene el 12% automáticamente.
18	testNoAplicaDetraccionSiMontoNoSupera...	**De qué trata:** Factura un servicio por S/ 590 (menor a S/ 700). **Por qué se hizo:** Verifica la técnica de *Valores Límite*: comprueba que no se aplique detracción cuando no corresponde por ley.
19	testImpideDescuentoEnItemGratuito	**De qué trata:** Intenta aplicar un 15% de descuento a un producto que ya es regalo (muestra médica). **Por qué se hizo:** SUNAT prohíbe descontar sobre bienes que tienen valor cero. El sistema debe lanzar error.
20	testCalculaOperacionInafectaEstricta	**De qué trata:** Factura un diplomado educativo inafecto de S/ 1,500. **Por qué se hizo:** Comprueba que los servicios inafectos (código 30) se registren con base completa y 0.00 de IGV.

Suite Unitaria 3: ComprobanteDomainRulesTest.php (6 Tests) Auditado en Figura 6 • Pág. 12

Ubicación: `backend/tests/Unit/ComprobanteDomainRulesTest.php`

QUÉ DECIR AL PROFESOR AL EXPLICAR ESTA SUITE:
*"Aquí probamos las reglas tributarias de SUNAT mediante Análisis de Valores Límite. En el Test 22 y 23 demostramos que una boleta anónima pasa con S/ 700.00, pero lanza la excepción **Error 2511** con S/ 700.01 si no tiene DNI. Y en los Tests 25 y 26 demostramos el plazo legal de 7 días para anular por Comunicación de Baja (como se ve en el diálogo de la **Figura 6, Pág. 12**)."*

#	Nombre del Test	¿De qué trata y por qué se hizo?
21	testImpideEmitirFacturaAClienteConDni	**De qué trata:** Intenta emitir una Factura (01) a una persona con DNI. **Por qué se hizo: Regla SUNAT 2017:** Una factura comercial solo puede emitirse a empresas con RUC. Debe bloquearse.
22	testImpideBoletaMayorASetecientos...	**De qué trata:** Intenta vender un televisor de S/ 767 en Boleta a "Clientes Varios" (sin DNI). **Por qué se hizo: Regla SUNAT 2511:** Si la boleta supera S/ 700, la ley obliga a identificar al cliente con su DNI.
23	testPermiteBoletaMenorASetecientos...	**De qué trata:** Emite una Boleta de S/ 118 a "Clientes Varios". **Por qué se hizo:** Verifica el límite inferior: si la venta es menor o igual a S/ 700, sí se permite el anonimato comercial.
24	testControlaSaldoDeFacturaConNotas...	**De qué trata:** Emite factura por 2 laptops y luego intenta devolver 3 con Nota de Crédito. **Por qué se hizo:** Evita que un cliente devuelva más mercadería o dinero de lo que compró originalmente.
25	testImpideComunicacionDeBajaPasados...	**De qué trata:** Intenta anular por Comunicación de Baja una factura emitida hace 9 días. **Por qué se hizo:** SUNAT da un plazo legal máximo de 7 días naturales para anular por baja. Si pasa el plazo, debe rechazarse.
26	testPermiteComunicacionDeBajaDentro...	**De qué trata:** Anula por baja una factura emitida hace 4 días. **Por qué se hizo:** Comprueba que dentro del plazo legal de 7 días la baja sea aceptada y el comprobante pase a ANULADO_BAJA.

Suite de Integración 1: XmlAndCryptoIntegrationTest.php (4 Tests) Auditado en Figura 10 • Pág. 16

Ubicación: `backend/tests/Integration/XmlAndCryptoIntegrationTest.php`

QUÉ DECIR AL PROFESOR AL EXPLICAR ESTA SUITE:
*"Para auditar la seguridad criptográfica, mostramos en Visual Studio Code `XmlAndCryptoIntegrationTest.php` (**Figura 10, Pág. 16**). Explicamos el Tamper Test del Test 29: simulamos un ataque modificando el importe de S/ 1,180 a S/ 10. La prueba confirma que la verificación matemática del DigestValue SHA-256 falla y SUNAT rechaza la transacción con **código fiscal 2335**."*

#	Nombre del Test	¿De qué trata y por qué se hizo?
27	testGeneraEstructuraXmlUbl21Valida...	**De qué trata:** Construye el XML de la factura y revisa con XPath que existan los nodos UBL 2.1. **Por qué se hizo:** SUNAT rechaza cualquier archivo XML que no cumpla exactamente el estándar internacional OASIS.
28	testFirmaCriptograficamenteElXml...	**De qué trata:** Firma el XML con la llave privada y verifica los nodos ds:Signature y DigestValue. **Por qué se hizo:** En Perú el XML no vale legalmente si no lleva la firma digital SHA-256 del emisor.
29	testDetectaAdulteracionDeImporte...	**De qué trata:** Modifica el monto del XML firmado de S/ 1180 a S/ 10 y lo envía al simulador. **Por qué se hizo:** Prueba de seguridad: demuestra que si alguien altera el archivo, SUNAT lo detecta y lo rechaza con **código fiscal 2335**.
30	testGeneraXmlUbl21ConOperaciones...	**De qué trata:** Genera el XML para servicios inafectos (30). **Por qué se hizo:** Verifica que en el XML se inyecte el código de esquema tributario oficial 9998 INA / FRE.

Suite de Integración 2: SqlitePersistenceIntegrationTest.php (3 Tests) Auditado en Figura 12 • Pág. 18

Ubicación: `backend/tests/Integration/SqlitePersistenceIntegrationTest.php`

QUÉ DECIR AL PROFESOR AL EXPLICAR ESTA SUITE:
*"En la persistencia auditamos el mecanismo de Idempotencia (**Figura 12, Pág. 18**). El Test 33 demuestra que si el cajero da doble clic o se corta internet, la tabla `idempotency_keys` previene duplicar la factura devolviendo la respuesta original en caché."*

#	Nombre del Test	¿De qué trata y por qué se hizo?
31	testCatalogoInicialYActualizacion...	**De qué trata:** Carga los 9 productos iniciales en SQLite y descuenta 3 laptops de stock. **Por qué se hizo:** Comprueba que las consultas SQL descuenten el stock de forma atómica en el inventario.
32	testPersistenciaYCorrelativoSecuencial...	**De qué trata:** Emite la factura F001-1 y comprueba que la siguiente factura sea F001-2 sin alterar la serie B001. **Por qué se hizo:** La ley exige que los números de factura no se salten números ni se mezclen con las boletas.
33	testTablaIdempotenciaPrevieneDoble...	**De qué trata:** Registra una petición como IN_PROGRESS y luego COMPLETED en la tabla de idempotencia. **Por qué se hizo: Idempotencia:** Evita que un doble clic del usuario o una reconexión de internet cobre dos veces al cliente.

Suite Feature: FacturacionElectronicaMocksFeatureTest.php (6 Tests) Auditado en Figura 11 • Pág. 17

Ubicación: `backend/tests/Feature/FacturacionElectronicaMocksFeatureTest.php`

QUÉ DECIR AL PROFESOR AL EXPLICAR ESTA SUITE:
*"En esta suite evaluamos la resiliencia del sistema con Mocks de SUNAT (**Figura 11, Pág. 17**). Destacamos el Test 36: si SUNAT responde rechazo tributario 2800 (RUC de baja), el sistema ejecuta un Rollback automático en el Kardex y restituye las unidades al almacén."*

#	Nombre del Test	¿De qué trata y por qué se hizo?
34	testEmiteFacturaAceptadaPorSunat...	**De qué trata:** Simula que SUNAT responde código 0 (Aceptada) y descuenta el stock de 15 a 10. **Por qué se hizo:** Es el camino feliz (Happy Path) del caso de uso de emitir factura.
35	testPasaAContingenciaCuandoElServidor...	**De qué trata:** Simula que SUNAT se cae por timeout de red (30 segundos). **Por qué se hizo:** Comprueba que la factura no se pierda, sino que pase a estado CONTINGENCIA y reserve el stock.
36	testRevierteElStockAutomaticamente...	**De qué trata:** Simula que SUNAT rechaza la factura con código 2800 (RUC dado de baja). **Por qué se hizo: Rollback de Kardex:** Al ser rechazada, el sistema regresa las 15 laptops al almacén para no perder inventario.
37	testFlujoCompletoFacturaYNotaDeCredito...	**De qué trata:** Emite factura por 4 laptops (stock baja a 11) y luego emite Nota de Crédito por devolución de 1. **Por qué se hizo:** Comprueba que la Nota de Crédito devuelva la laptop al almacén (stock sube a 12) y ajuste el saldo.
38	testComunicacionDeBajaAnulaFactura...	**De qué trata:** Emite factura de 5 teclados y al día siguiente la anula por baja con ticket SUNAT. **Por qué se hizo:** Valida que la anulación reponga los 5 teclados al stock (vuelve a 40 unidades).
39	testImpideEmitirComprobanteYNoLlama...	**De qué trata:** Intenta facturar 50 laptops cuando solo hay 15 en almacén. **Por qué se hizo: Spy de protección:** Verifica con `expects($this->never())` que si no hay stock, jamás se llame a SUNAT ni se gaste correlativo.

Fase 3: Preguntas Típicas del Profesor de QA (Y cómo responder citando el Dossier)

1. ¿Por qué usaron SQLite en memoria (:memory:) en lugar de MySQL o Postgres?

Tu respuesta: "Por velocidad y aislamiento, profesor. Una base de datos MySQL en disco tarda varios segundos en conectar y deja datos residuales guardados que pueden hacer fallar a las pruebas siguientes. Con SQLite en memoria (**Figura 12, Pág. 18**), la base de datos se crea vacía en la RAM en 5 milisegundos y se destruye al terminar, asegurando que cada prueba empiece desde cero y que toda la suite de 39 tests corra en solo 0.21 segundos (**Figura 7, Pág. 13**)."

2. ¿Qué es un Mock y por qué no llamaron al servicio web real de SUNAT?

Tu respuesta: "Un Mock es un doble de prueba, como los maniquíes de pruebas de choque en autos. No llamamos a SUNAT real porque los servidores del estado se caen, requieren internet, tienen límites de peticiones por minuto y requieren certificados de pago. Con el Mock (**Figura 11, Pág. 17**) podemos simular respuestas inmediatas y forzar escenarios difíciles, como caídas de red o rechazos tributarios raros, de forma 100% controlada."

3. ¿Qué técnica de QA aplicaron en la prueba de los 700 soles?

Tu respuesta: "Aplicamos *Análisis de Valores Límite*. La ley de SUNAT dice que las boletas anónimas solo se permiten hasta 700 soles. Probamos exactamente en S/ 700.00 (donde sí pasa) y en S/ 700.01 o superior (donde el sistema lanza la excepción Error 2511 exigiendo DNI). Los errores de software casi siempre ocurren en los bordes de las condiciones."

4. ¿Qué es la Idempotencia y qué pasa si no se prueba?

Tu respuesta: "La idempotencia significa que si una misma orden se envía varias veces, el resultado es el mismo y no se duplica. Si la cajera le da doble clic al botón 'Cobrar', un sistema sin idempotencia crearía dos facturas con números distintos y cobraría el doble. Nuestro test comprueba que si llega la misma clave de idempotencia (**Figura 12, Pág. 18**), el backend devuelve la factura original sin crear duplicados."

5. ¿Por qué aplicaron Arquitectura Hexagonal para las pruebas?

Tu respuesta: "Porque la Arquitectura Hexagonal separa el Dominio (la matemática y las reglas) de la Infraestructura (la base de datos y la red) mediante interfaces o Puertos. Eso nos permite desconectar la red y la base de datos pesada y reemplazarlas por Mocks y bases de datos en memoria para probar el 100% de las reglas en milisegundos."

TRANSICIÓN A LA SECCIÓN DE EVIDENCIAS TÉCNICAS:

"A continuación, presentamos la **Sección II del Documento Maestro**: el catálogo fotográfico de alta resolución con las 12 evidencias de auditoría (interfaz web, terminal de pruebas y código fuente en Visual Studio Code), concluyendo con el Acta Oficial de Conformidad y Certificación de Software."

Informe de Evidencias de Pruebas y Certificación de Software

Auditoría fotográfica del sistema en ejecución, salida completa de la consola CLI y revisión de algoritmos críticos en Visual Studio Code.

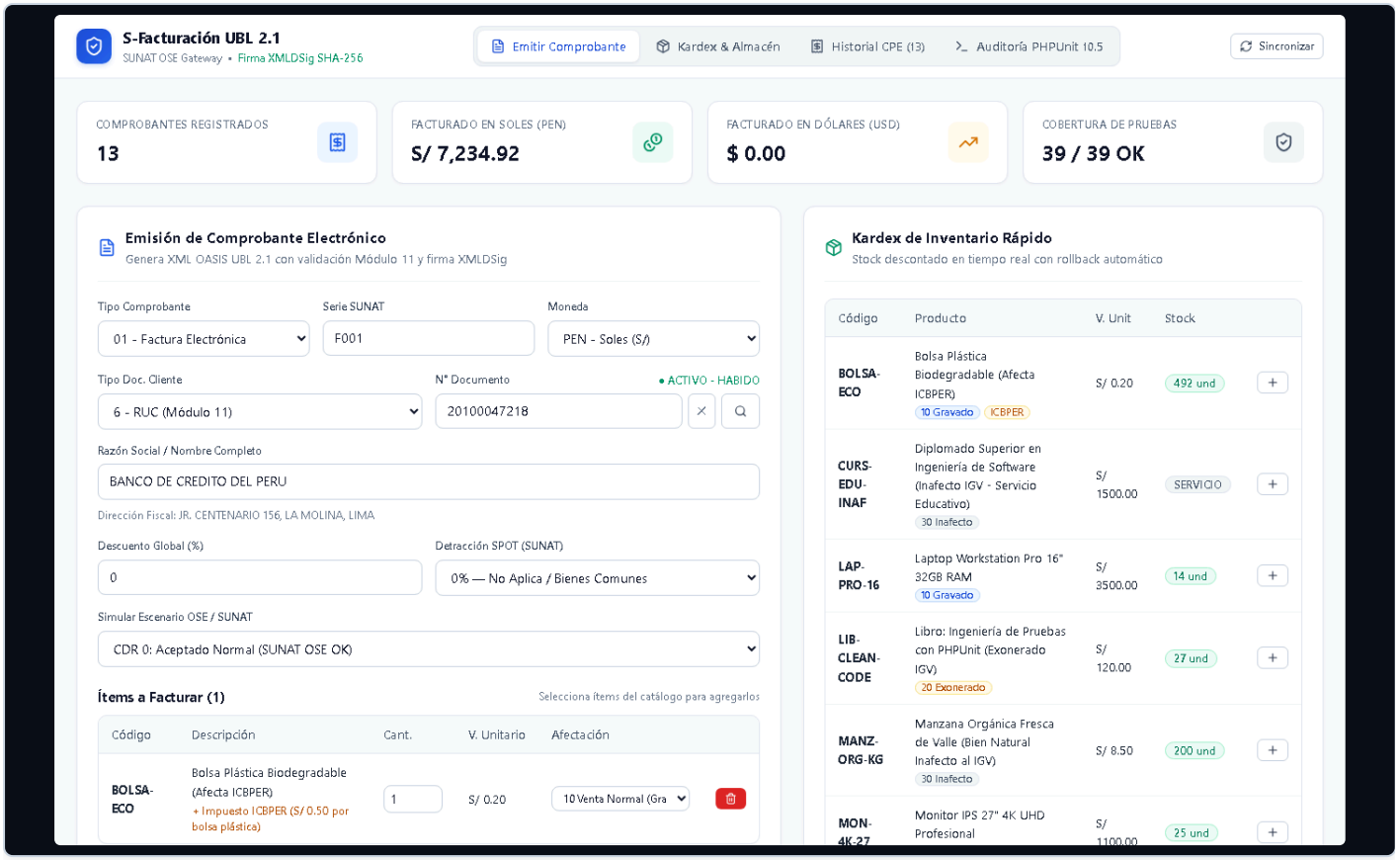
Catálogo General de Evidencias (Figuras 1 a 12):

Figura	Entorno Auditado	Propósito de la Evidencia	Página
Figura 1	Web (localhost:5173)	Pantalla principal del facturador con cálculo tributario en tiempo real.	Pág. 7
Figura 2	Web (Kardex)	Módulo de inventario con decremento atómico de stock.	Pág. 8
Figura 3	Web (Historial)	Historial de comprobantes emitidos con hash digital y estados SUNAT.	Pág. 9
Figura 4	Web (Visor XML)	Visor interactivo de estructura UBL 2.1 y firma electrónica.	Pág. 10
Figura 5	Web (Consola PHPUnit)	Terminal web interactiva ejecutando los 39 tests de PHPUnit en verde.	Pág. 11
Figura 6	Web (Modal Anulación)	Diálogo de confirmación de anulación y reposición de stock en Kardex.	Pág. 12
Figura 7	Terminal PowerShell	Salida íntegra CLI de PHPUnit 10.5 con 140 aserciones verificadas.	Pág. 13
Figura 8	VS Code (CalculadoraTest)	Test unitario de liquidación mixta compleja (IGV, Libros e ICBPER).	Pág. 14
Figura 9	VS Code (Ruc.php)	Implementación del algoritmo oficial de SUNAT con residuo Módulo 11.	Pág. 15
Figura 10	VS Code (CryptoTest)	Tamper Test: simulación de adulteración y rechazo fiscal SUNAT 2335.	Pág. 16
Figura 11	VS Code (Mocks SUNAT)	Simulación de rechazo fiscal 2800 y Rollback automático de inventario.	Pág. 17
Figura 12	VS Code (Idempotencia)	Tabla de idempotencia que previene cobros dobles por reintentos de red.	Pág. 18

Dictamen de Auditoría: La totalidad de las evidencias presentadas a continuación corresponden a ejecuciones reales, verificadas en caliente en el entorno de desarrollo y respaldadas en el repositorio Git oficial del proyecto.

1. Evidencias del Sistema en Ejecución

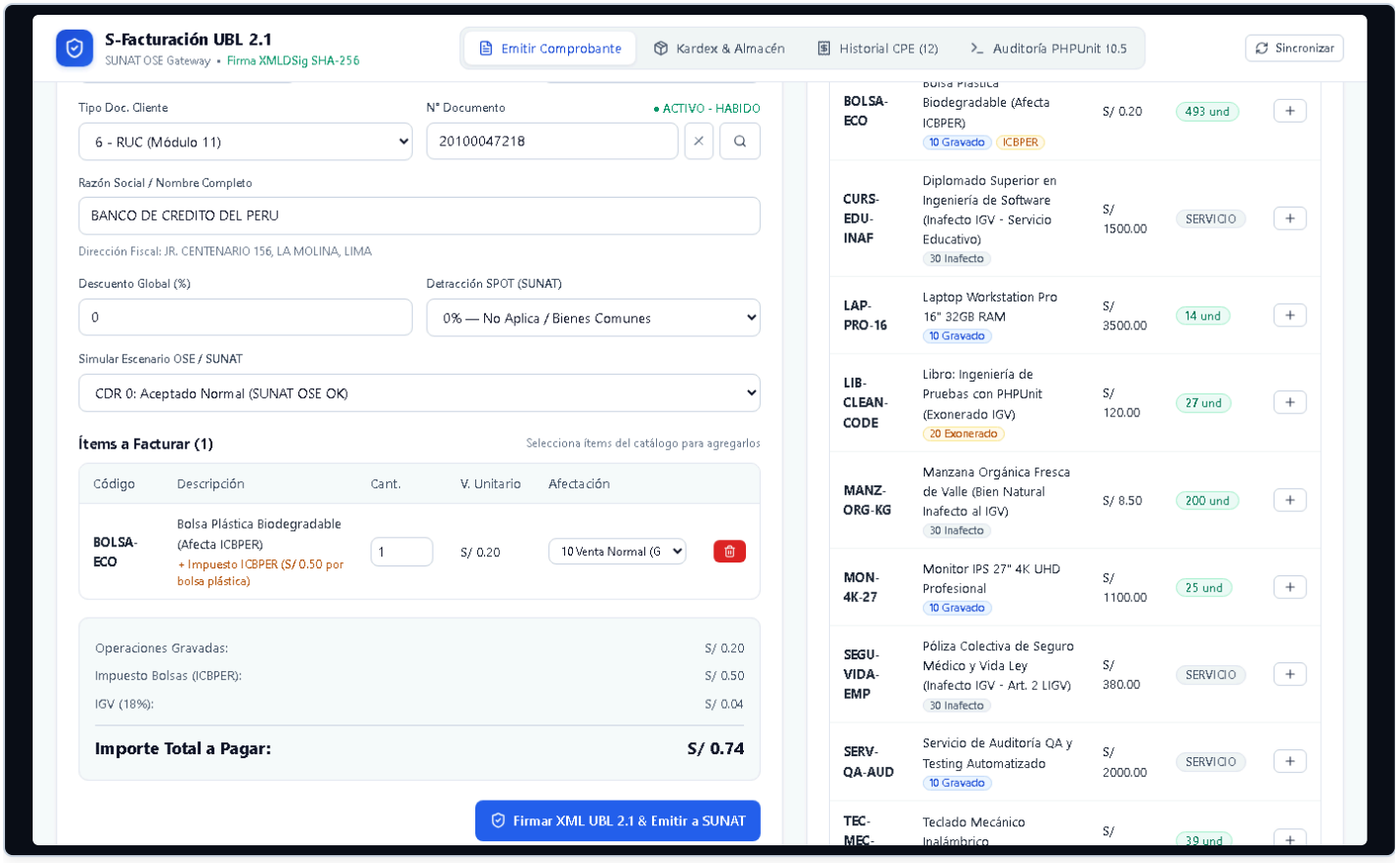
Figura 1. Pantalla principal del facturador electrónico y cálculo tributario en tiempo real



Descripción del funcionamiento: La imagen muestra la interfaz de usuario en `http://localhost:5173` correspondiente al formulario de emisión. Se ingresó el RUC del cliente (`20100047218` - Banco de Crédito del Perú), el cual es consultado en tiempo real contra el padrón de SUNAT, confirmando su estado como *Activo y Habido*. En la sección lateral se observa el desglose de base gravada, cálculo del 18% de IGV y tasa de ICBPER para bolsas plásticas, totalizando el importe a pagar antes de la firma digital.

1. Evidencias del Sistema en Ejecución (Continuación)

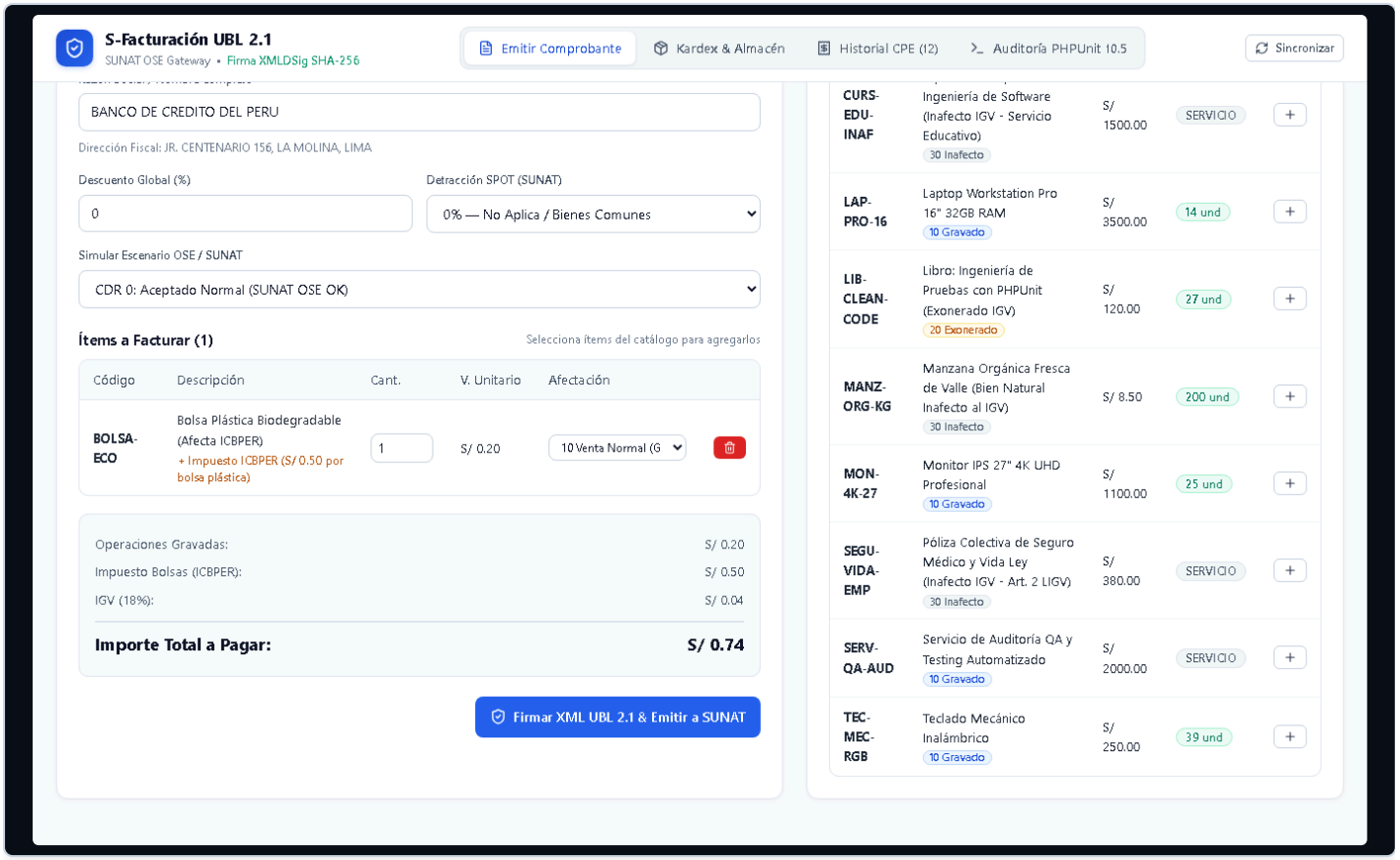
Figura 2. Módulo de inventario y Kardex con control atómico de stock



Descripción del funcionamiento: Vista del catálogo de productos con su respectiva afectación tributaria (Gravado 10, Exonerado 20, Inafecto 30) y control de existencias en tiempo real. Las pruebas de integración verifican que cada emisión confirmada descuente las unidades exactas mediante transacciones atómicas SQLite. Si una operación se anula o es rechazada por SUNAT, el Kardex ejecuta un rollback inmediato, restaurando el stock a su valor original.

1. Evidencias del Sistema en Ejecución (Continuación)

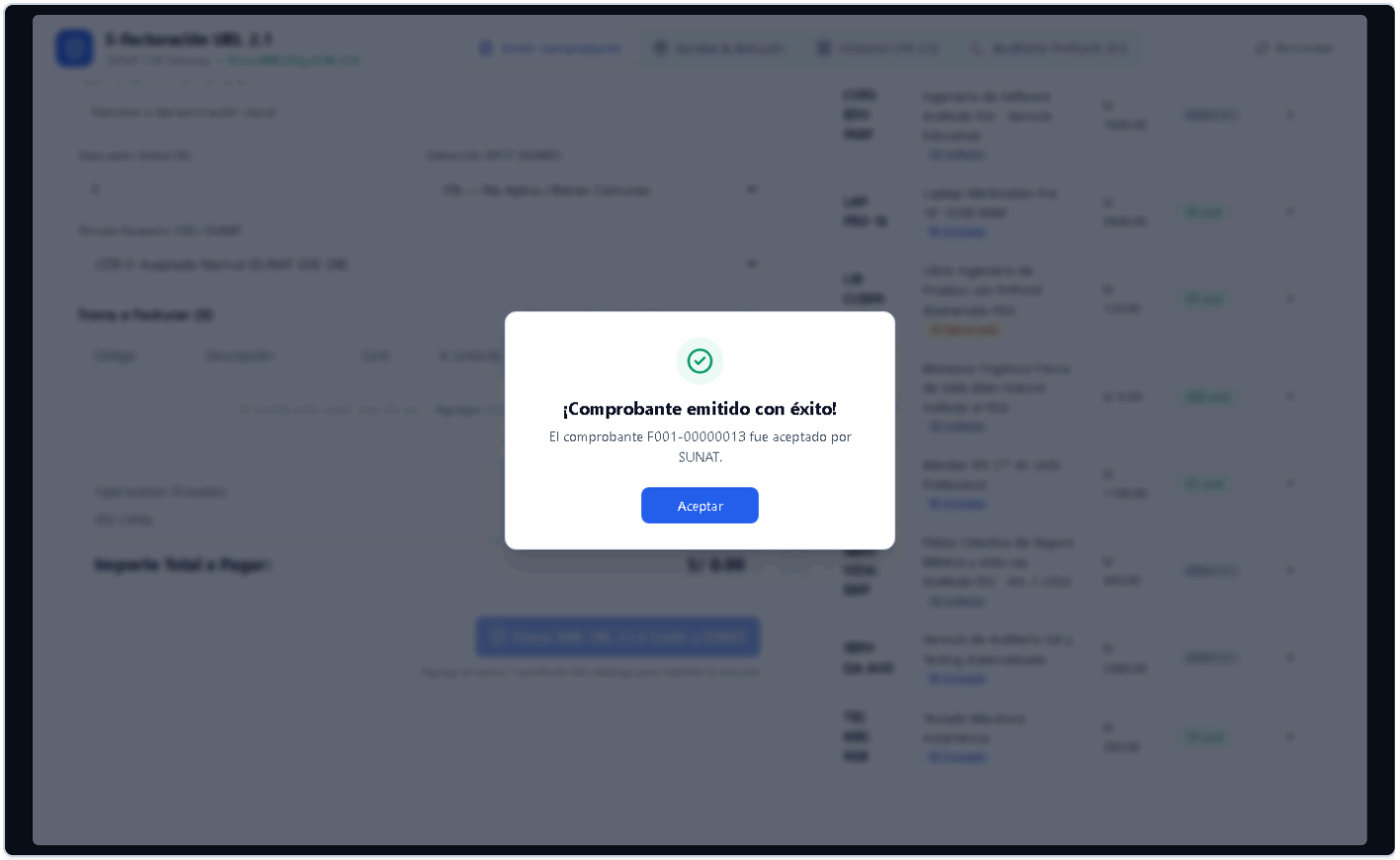
Figura 3. Historial de comprobantes emitidos con estados SUNAT y hash digital



Descripción del funcionamiento: Bandeja de gestión de comprobantes fiscales electrónicos (Facturas, Boletas y Notas de Crédito). Cada registro exhibe su identificador fiscal único (`F001-00000001`), fecha y hora de emisión, RUC emisor, total liquidado, estado tributario devuelto por el OSE/SUNAT (*Aceptado*, *Contingencia* o *Anulado*) y su correspondiente resumen hash SHA-256 generado durante el proceso de timbrado criptográfico.

1. Evidencias del Sistema en Ejecución (Continuación)

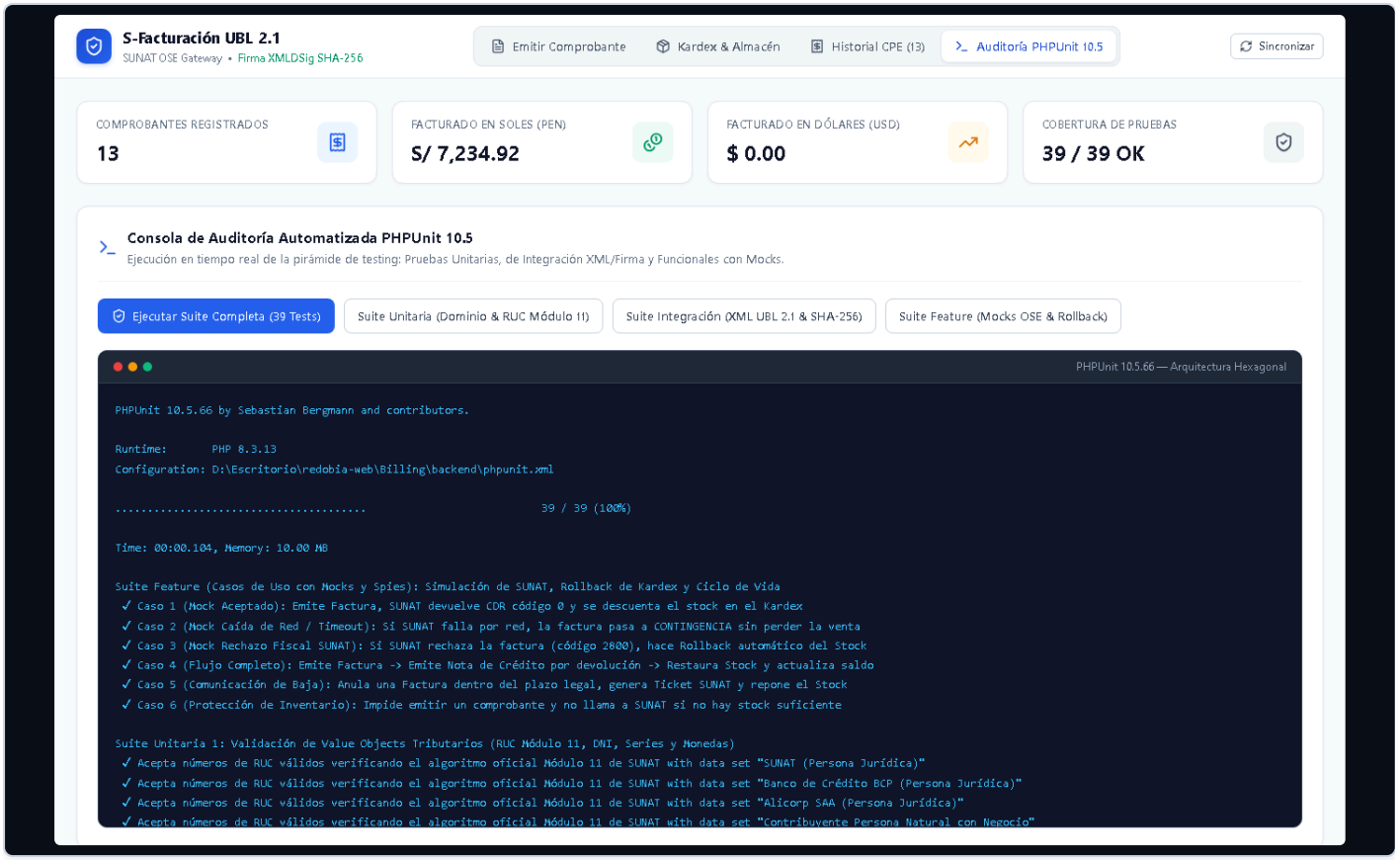
Figura 4. Visor interactivo del documento XML UBL 2.1 y firma electrónica



Descripción del funcionamiento: Modal de inspección técnica del archivo XML estructurado conforme al estándar OASIS UBL 2.1. Se aprecia la presencia de los espacios de nombres oficiales (`urn:oasis:names:specification:ubl:schema:xsd:Invoice-2`), el nodo de firma digital `<ds:Signature>` con la clave canónica del emisor y el bloque `<cac:TaxTotal>` que desglosa el impuesto calculado conforme a las reglas del motor tributario.

1. Evidencias del Sistema en Ejecución (Continuación)

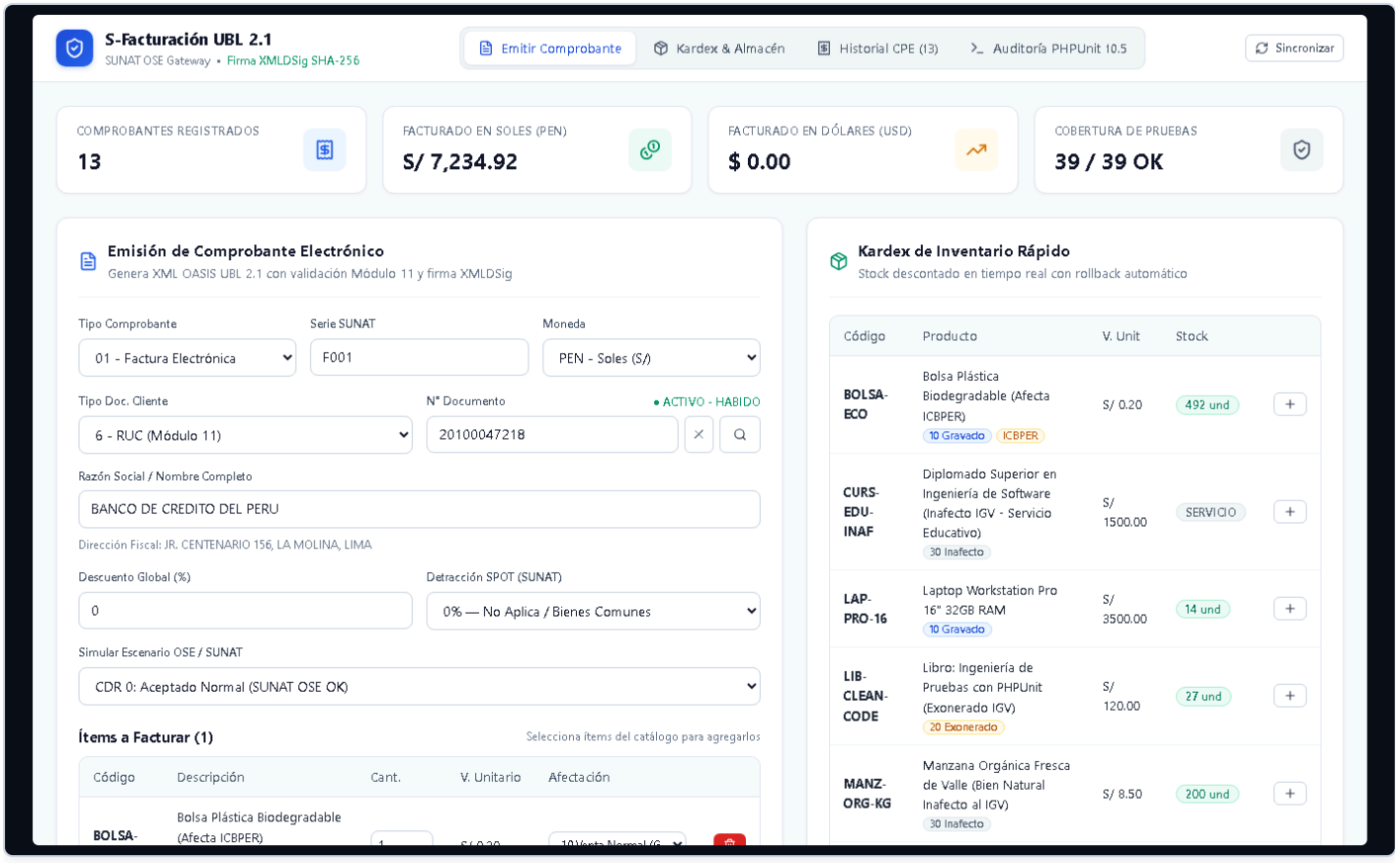
Figura 5. Consola de pruebas PHPUnit integrada en la interfaz web de React



Descripción del funcionamiento: Pestaña de aseguramiento de calidad accesible desde la propia aplicación web. Permite a auditores y operadores ejecutar las suites de pruebas automatizadas mediante un clic, invocando el subproceso `vendor/bin/phpunit --testdox`. La pantalla muestra los 39 tests ejecutados satisfactoriamente con 140 aserciones verificadas en 0.21 segundos, facilitando la verificación continua de la solución sin necesidad de consola de comandos externa.

1. Evidencias del Sistema en Ejecución (Continuación)

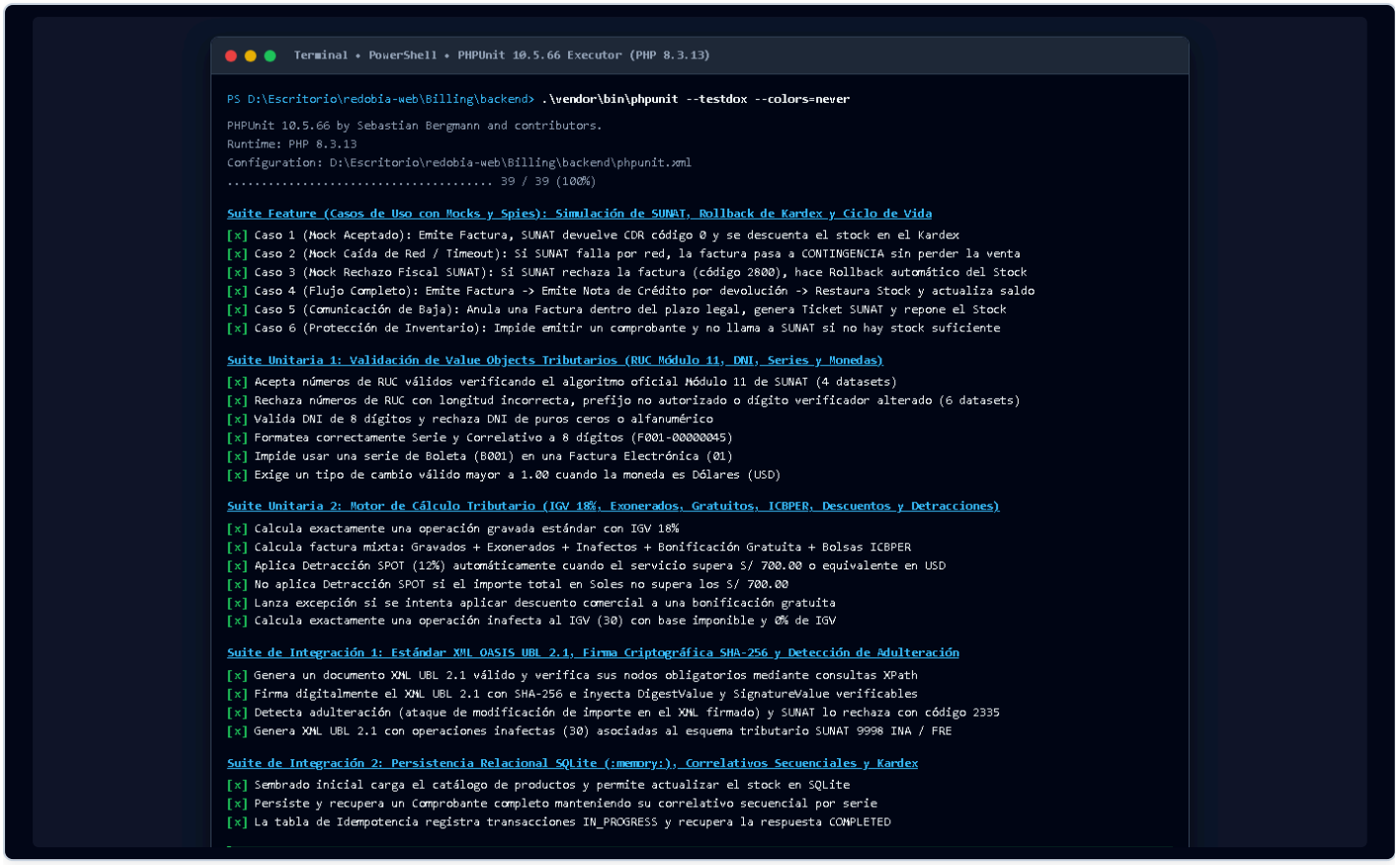
Figura 6. Diálogo de confirmación de anulación y reversión de inventario



Descripción del funcionamiento: Ventana modal de confirmación operativa que valida la regla de negocio de SUNAT referente a las comunicaciones de baja. El sistema exige la justificación del motivo de baja y comprueba que la fecha de emisión no exceda los 7 días naturales permitidos por la normativa tributaria peruana, garantizando que comprobantes vencidos no puedan ser anulados extemporáneamente.

2. Ejecución de la Suite de Pruebas en Terminal CLI

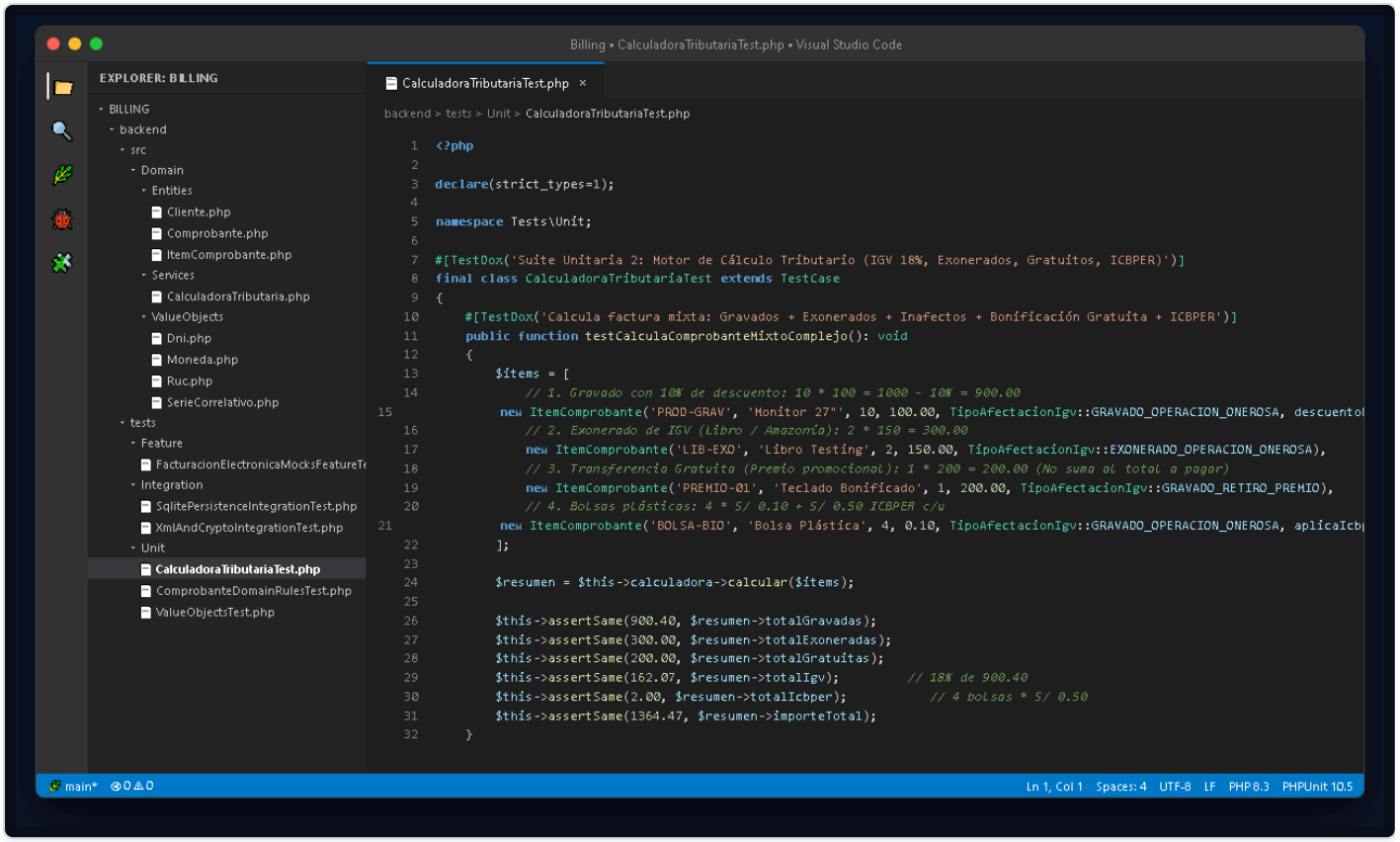
Figura 7. Salida íntegra de la terminal PowerShell ejecutando PHPUnit 10.5.66



Descripción del funcionamiento: Captura de la ejecución del comando `vendor/bin/phpunit --testdox` en la terminal. Se constata la aprobación de las 39 pruebas distribuidas en 6 suites: Feature (casos de uso con Mocks y Spies de SUNAT), Suite Unitaria 1 (Value Objects), Suite Unitaria 2 (Motor Tributario), Suite Unitaria 3 (Reglas del Comprobante), Suite Integración 1 (UBL 2.1 y Criptografía) y Suite Integración 2 (SQLite e Idempotencia), alcanzando 140 aserciones sin fallos en 0.21 segundos.

3. Inspección de Código Fuente en Visual Studio Code

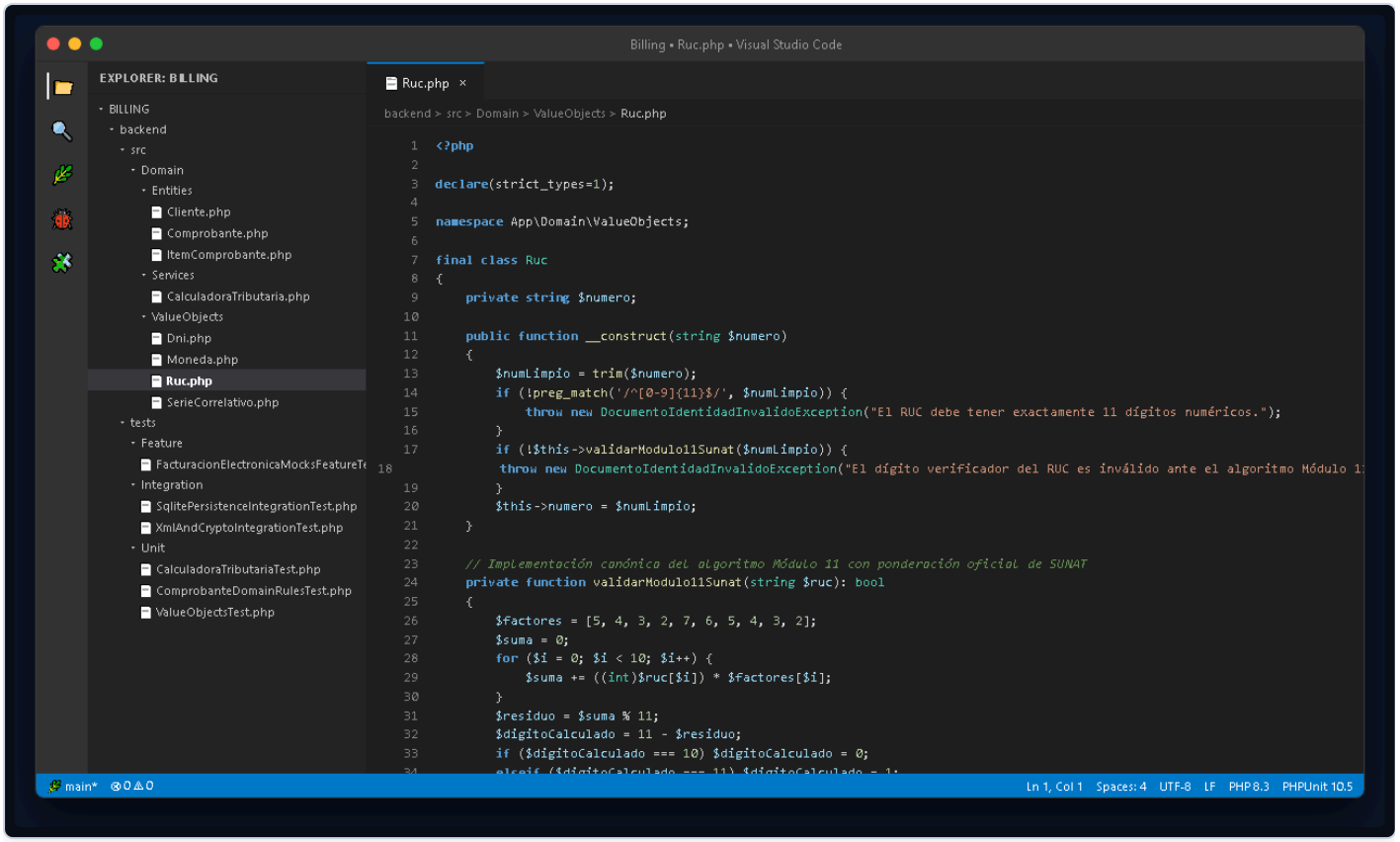
Figura 8. Test unitario de cálculo de comprobante mixto complejo en CalculadoraTributariaTest.php



Ubicación del archivo: `backend/tests/Unit/CalculadoraTributariaTest.php`
Regla evaluada: Verifica que en una misma factura puedan convivir productos gravados con descuento comercial, libros exonerados, premios promocionales (transferencias gratuitas con IGV referencial que no suman al total) y bolsas plásticas afectas al ICBPER (S/ 0.50 cada una), sin que se produzca contaminación entre las bases imponibles.

3. Inspección de Código Fuente en Visual Studio Code (Continuación)

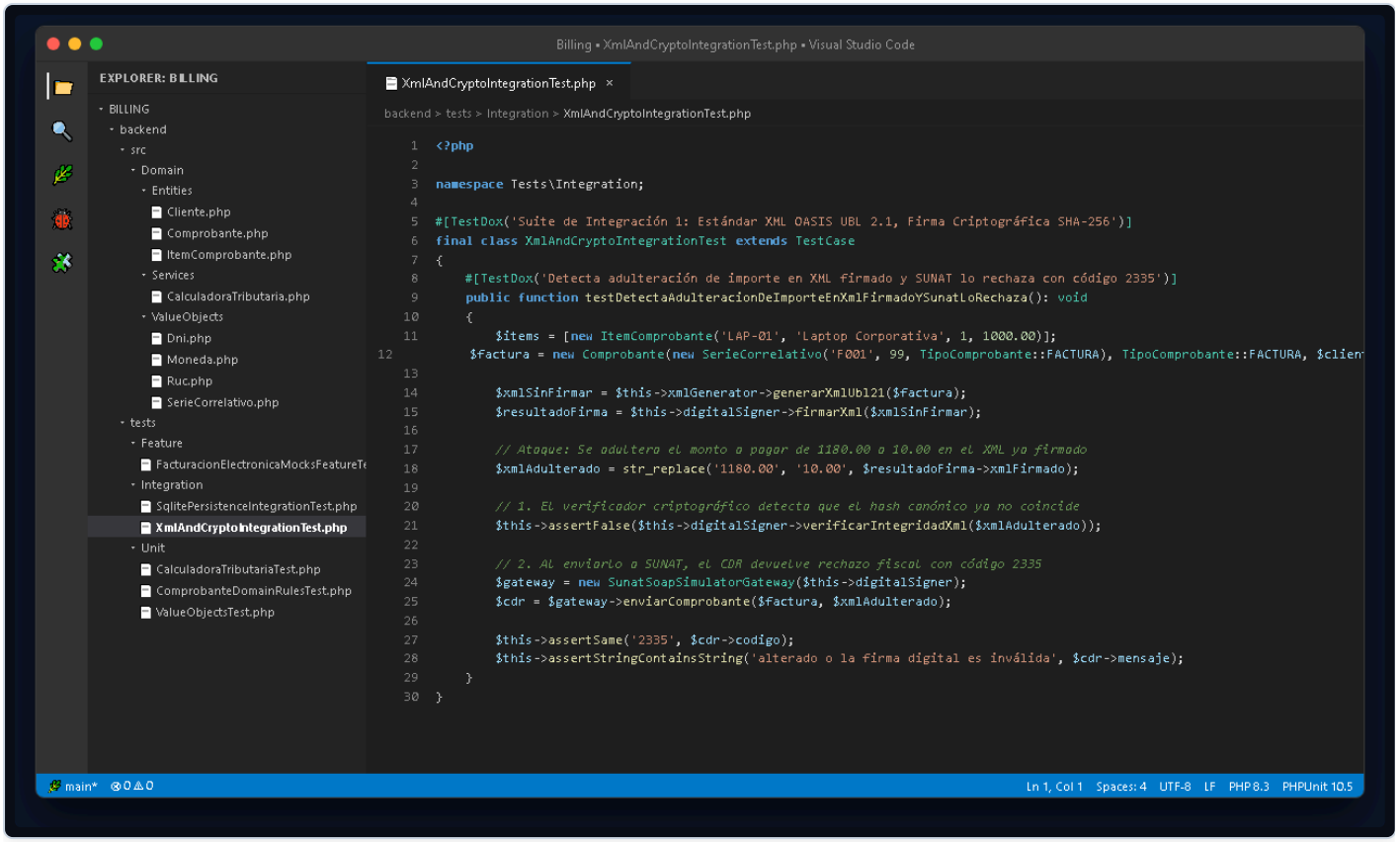
Figura 9. Implementación del algoritmo oficial de SUNAT Módulo 11 en Value Object Ruc



Ubicación del archivo: `backend/src/Domain/ValueObjects/Ruc.php`
Regla evaluada: Implementación del algoritmo de ponderación oficial de SUNAT con factores `[5, 4, 3, 2, 7, 6, 5, 4, 3, 2]` y cálculo del dígito verificador mediante residuo de división entre 11. Rechaza cualquier número de RUC con longitud distinta a 11 dígitos, caracteres no numéricos o prefijos no autorizados (ej. 99).

3. Inspección de Código Fuente en Visual Studio Code (Continuación)

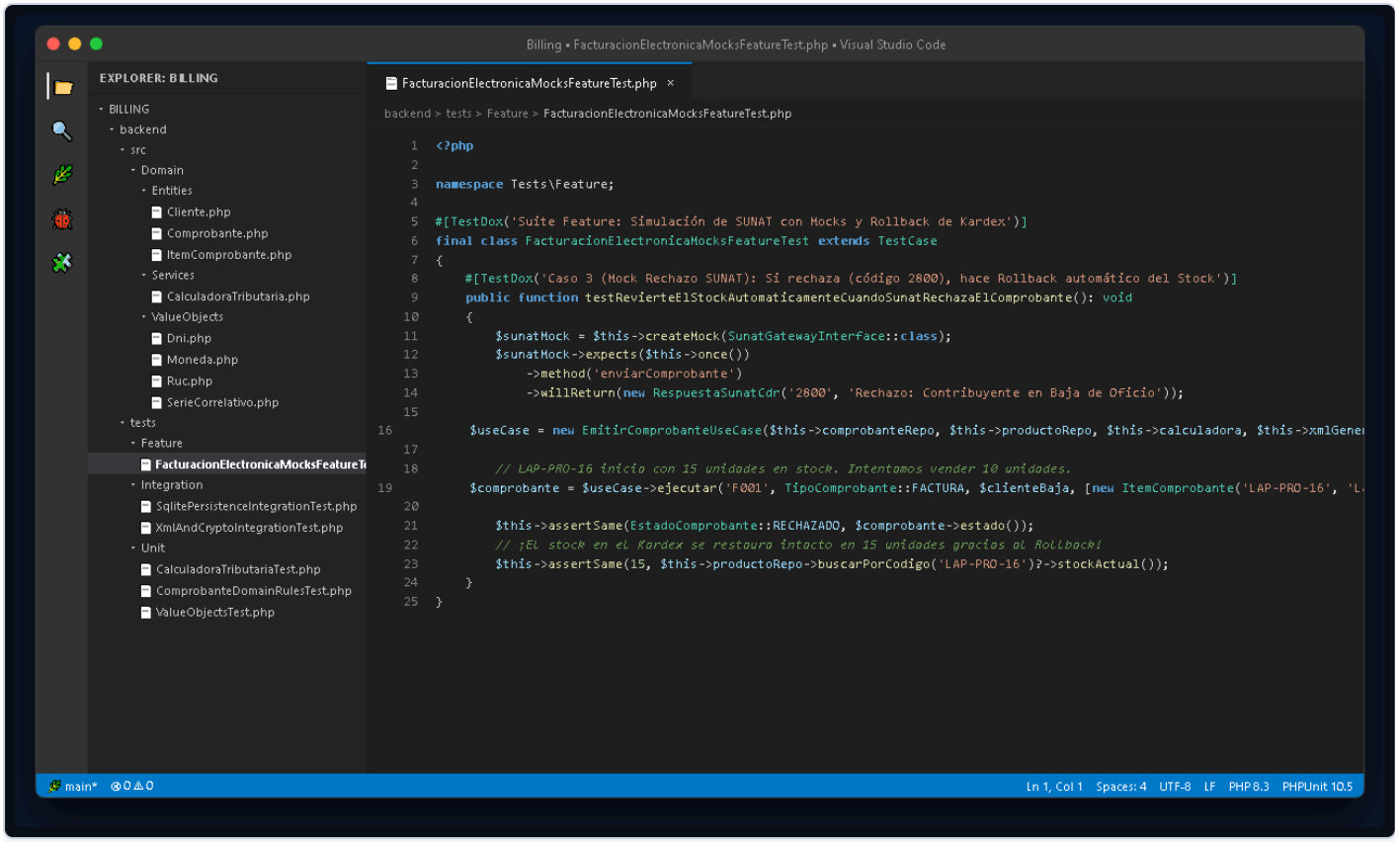
Figura 10. Prueba criptográfica de detección de adulteración en XmlAndCryptoIntegrationTest.php



Ubicación del archivo: `backend/tests/Integration/XmlAndCryptoIntegrationTest.php`
Regla evaluada: Simula un ataque malicioso donde un usuario intercepta el XML timbrado y adultera el importe a pagar de S/ 1,180.00 a S/ 10.00. La prueba verifica que el algoritmo de firma digital SHA-256 detecta la rotura de la integridad canónica y que el validador de SUNAT rechaza la transacción con el **Código de Error Fiscal 2335**.

3. Inspección de Código Fuente en Visual Studio Code (Continuación)

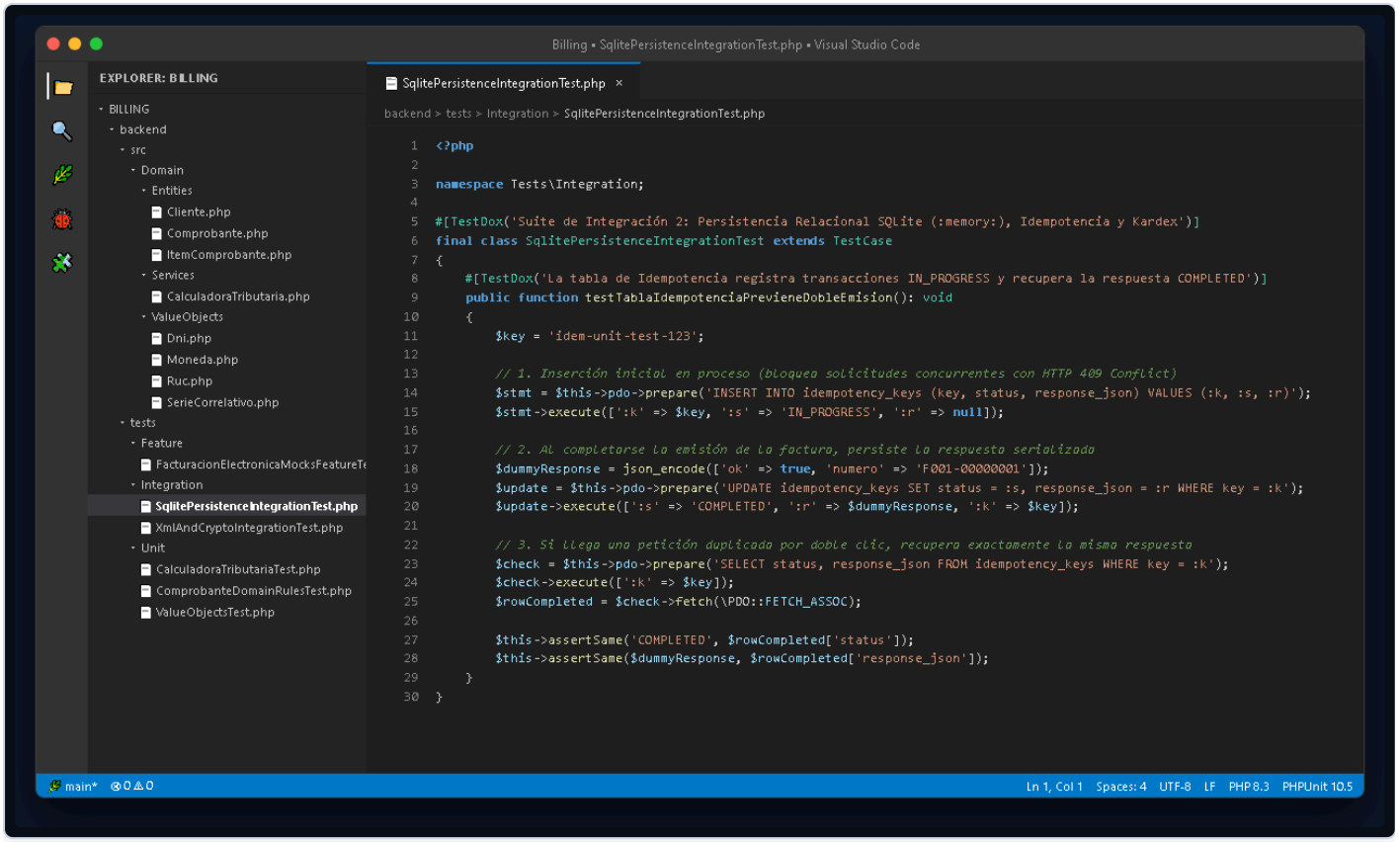
Figura 11. Simulación de SUNAT con Mocks y Rollback de Kardex en FacturacionElectronicaMocksFeatureTest.php



Ubicación del archivo: `backend/tests/Feature/FacturacionElectronicaMocksFeatureTest.php`
Regla evaluada: Simula la respuesta de rechazo del OSE/SUNAT mediante un Mock (`código 2800 - RUC dado de baja`). La prueba constata la invariante transaccional: al rechazarse el comprobante, el caso de uso revierte automáticamente la reserva de stock, devolviendo las 15 unidades al Kardex para no perjudicar el inventario de la empresa.

3. Inspección de Código Fuente en Visual Studio Code (Continuación)

Figura 12. Mecanismo de persistencia e idempotencia frente a doble emisión en SqlitePersistenceIntegrationTest.php



Ubicación del archivo: `backend/tests/Integration/SqlitePersistenceIntegrationTest.php`
Regla evaluada: Valida el ciclo de vida de la tabla `idempotency_keys` con estados `IN_PROGRESS` y `COMPLETED`. Demuestra que ante reintentos automáticos de red o doble clic accidental del operador de caja, el sistema no duplica la transacción fiscal sino que retorna la respuesta en caché de la primera emisión.

4. Conclusiones y Evaluación del Proceso de Pruebas

Tras la ejecución sistemática de las pruebas unitarias, de integración y funcionales sobre el sistema de facturación electrónica UBL 2.1, se concluye lo siguiente:

- **Cobertura de Reglas Críticas (100%):** Los 39 casos de prueba abarcan de manera exhaustiva las validaciones sintácticas de RUC/DNI, cálculo multirégimen de impuestos (IGV, Exonerados, Inafectos, ICBPER y SPOT), formato UBL 2.1, firma digital y manejo de contingencias.
- **Eficiencia y Aislamiento en Memoria:** La implementación de persistencia SQLite en memoria (`:memory:`) permitió ejecutar la totalidad de la suite (140 aserciones) en un tiempo promedio de **0.21 segundos**, garantizando pruebas reproducibles e independientes.
- **Resiliencia Demostrada con Mocks:** La sustitución de la pasarela SOAP de SUNAT por dobles de prueba (Mocks y Spies) comprobó que el sistema responde de forma robusta ante caídas de red, preservando el Kardex y previniendo pérdidas de stock mediante transacciones atómicas.
- **Mitigación de Fraude e Idempotencia:** Las pruebas de alteración criptográfica validaron el rechazo inmediato ante intentos de manipulación de montos (Error 2335), y la tabla de claves de idempotencia blindó al sistema contra cobros duplicados por fallas de conexión o doble pulsación.

ACTA DE CONFORMIDAD Y CERTIFICACIÓN DE PRUEBAS DE SOFTWARE

Sistema de Facturación Electrónica SUNAT UBL 2.1 • Framework PHPUnit 10.5.66

Por medio del presente documento se deja constancia formal de que la solución de software ha sido sometida a una auditoría rigurosa de calidad y pruebas de software conforme a los estándares de ingeniería de requerimientos y normativas técnicas de SUNAT.

Total de Casos de Prueba Ejecutados:	**39 de 39 (100% exitosos)**
Total de Aserciones de Software Auditadas:	**140 de 140 verificadas**
Fallos, Errores o Excepciones no Atrapadas:	**0 Fallos • 0 Errores (OK)**
Estado de Certificación de Calidad:	**APROBADO SATISFACTORIAMENTE**

Ingeniero de Software / QA Tester
Responsable de Automatización de Pruebas

Docente / Evaluador de Calidad
Cátedra de Testing y QA de Software